



兰州工业学院

实 验 报 告

(2024—2025学年第1学期)

课程名称 嵌入式系统应用

专业班级 专升本电信 23

学 号 ZB2305010116

姓 名 姜 勇

实验项目	实验一 Linux下C语言程序编译及调试		
实验时间	2024. 09. 21	实验地点	公共教学楼B101
一、实验内容 1. 掌握Ubuntu环境下GCC编译器的安装。 2. 掌握Ubuntu环境下C程序的编辑和GCC编译。 3. 掌握Ubuntu环境下C程序的GDB调试方法。 二、实验器材（设备、元器件） 1. 个人计算机一台 2. Ubuntu Linux操作系统 3. 计算机可以接入互联网 三、实验步骤 1. 使用C语言编写一个change()函数实现输入字符串的大小写转换功能，把该函数加入libxxx.so库（xxx是自己的姓名全拼音）文件，将库文件保存到/home/test/libxx（xx是学号后两位）目录下，最后写一个main.c来调用库文件中的change()函数。 1.1 程序设计的流程图			

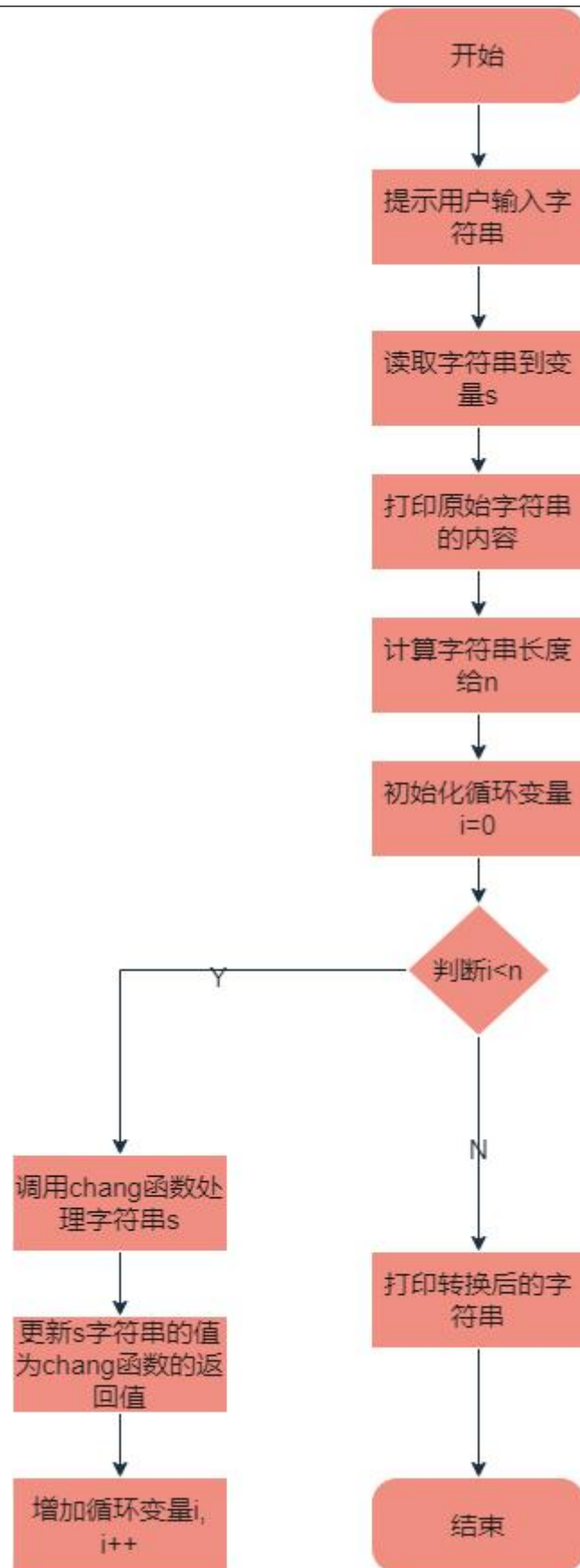


图 1 程序流程图

1.2 程序设计的关键代码

Chang函数

```
char chang(char ch)
{
    if(ch>='A' &&ch<='Z')
    {
        ch=ch+32;
    }else if(ch>='a' &&ch<='z')
    {
        ch=ch-32;
    }
    return ch;
}
```

main.c

```
#include <stdio.h>
#include <string.h>
char chang(char ch);
int main() {
    char s[100];
    int i,n;
    printf("请输入一个字符串：");
    scanf("%s", s);
    printf("原始字符串： %s\n",s);
    n=strlen(s);
    for(i=0;i<n;i++)
    {
        s[i]=chang(s[i]);
    }
    printf("转换之后的字符串： %s\n",s);
}
```

```

    return 0;
}

2. 使用GDB调试完成程序的调试过程

2.1 完成课本85页的GDB的基本使用实例

代码：

#include<stdio.h>

int sum(int n);

main()
{
    int s=0;
    int i,n;
    for(i=0;i<=50;i++)
    {
        s=i+s;
    }
    s=s+sum(20);
    printf("the result is  %d\n",s);
}

int sum(int n)
{
    int total=0;
    int i;
    for(i=0;i<=n;i++)
        total=total+i;
    return (total);
}

```

```
JY5@JY5:~/Desktop/test/test1$ gcc test_g.c -g -o test_g
test_g.c:3:1: warning: return type defaults to 'int' [-Wimplicit-int]
main()
^~~~
JY5@JY5:~/Desktop/test/test1$
```

图 2 编译test_g.c生成一个带有调试信息的可执行文件 test_g

```
JY5@JY5:~/Desktop/test/test1$ gdb test_g
GNU gdb (Gdb 8.2.1.1-1+security) 8.2.1
Copyright (C) 2018 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from test_g...done.
(gdb)
```

图 3 启动GDB调试环境

```
(gdb) 1
1      #include<stdio.h>
2      int sum(int n);
3      main()
4      {
5          int s=0;
6          int i,n;
7          for(i=0;i<=50;i++)
8          {
9              s=i+s;
10         }
(gdb)
```

图 4 输入1查看源代码，默认10行

```
(gdb) break 7
Breakpoint 1 at 0x401131: file test_g.c, line 7.
(gdb)
```

图 5 在源代码第7行设置断点

```
(gdb) break sum
Breakpoint 2 at 0x401179: file test_g.c, line 16.
(gdb)
```

图 6 在源代码sum处设置断点

```
(gdb) info break
Num      Type           Disp Enb Address          What
1        breakpoint     keep y   0x0000000000401131 in main at test_g.c:7
2        breakpoint     keep y   0x0000000000401179 in sum at test_g.c:16
(gdb)
```

图 7 显示断点信息

```
(gdb) r
Starting program: /home/JY5/Desktop/test/test1/test_g

Breakpoint 1, main () at test_g.c:7
7          for(i=0;i<=50;i++)
(gdb)
```

图 8 运行程序

```
(gdb) n
9                               s=i+s;
(gdb) █
```

图 9 在第一个断点处停止，n单步执行

```
(gdb) print s
$1 = 0
(gdb) █
```

图 10 输出变量s的值

```
(gdb) c
Continuing.

Breakpoint 2, sum (n=20) at test_g.c:16
16          int total=0;
(gdb) █
```

```
(gdb) c
Continuing.
the result is 1485
[Inferior 1 (process 17358) exited normally]
(gdb)
```

图 11 输入c，继续执行到程序结束

```
(gdb) q
JY5@JY5: ~/Desktop/test/test1$ █
```

图 12 退出gdb

2.2完成课本88页GDB典型实例的编译、程序执行和调试过程代码：

```
#include<stdio.h>
#include<string.h>
int main(void)
{
    int i,len;
    char str[]="hello";
    char *rev_string;
    len=strlen(str);
    rev_string=(char *)malloc(len+1);
    printf("%s\n",str);
    for(i=0;i<len;i++)
        rev_string[len-i]=str[i];
```

```

rev_string[len+1]='\0';

printf("the reverse string is%s\n",rev_string);

}

```

编译、执行

```

JY5@JY5:~/Desktop/test/test1$ gcc bug.c -o bug
bug.c: In function 'main':
bug.c:9:21: warning: implicit declaration of function 'malloc' [-Wimplicit-function-declaration]
   rev_string=(char *)malloc(len+1);
                       ^~~~~~
bug.c:9:21: warning: incompatible implicit declaration of built-in function 'malloc'
bug.c:9:21: note: include '<stdlib.h>' or provide a declaration of 'malloc'
bug.c:3:1:
+#include <stdlib.h>
int main(void)
bug.c:9:21:
   rev_string=(char *)malloc(len+1);
                       ^~~~~~
JY5@JY5:~/Desktop/test/test1$ ./bug
hello
the reverse string is
JY5@JY5:~/Desktop/test/test1$

```

图 13 编译bug.c生成可执行文件，运行这个文件

调试

```

(gdb) l
1      #include<stdio.h>
2      #include<string.h>
3      int main(void)
4      {
5          int i,len;
6          char str[]="hello";
7          char *rev_string;
8          len=strlen(str);
9          rev_string=(char *)malloc(len+1);
10         printf("%s\n",str);
(gdb) l
11         for(i=0;i<len;i++)
12             rev_string[len-i]=str[i];
13         rev_string[len+1]='\0';
14         printf("the reverse string is%s\n",rev_string);
15     }
16

```

图 14 输入l查看源代码

```

(gdb) break 8
Breakpoint 1 at 0x401167: file bug.c, line 8.
(gdb)

```

图 15 在源代码第8行设置断点

```

(gdb) n
9          rev_string=(char *)malloc(len+1);
(gdb) print len
$1 = 5
(gdb) n
10         printf("%s\n",str);
(gdb) n
hello
11         for(i=0;i<len;i++)
(gdb) n
12             rev_string[len-i]=str[i];
(gdb) n
11         for(i=0;i<len;i++)
(gdb) n
12             rev_string[len-i]=str[i];
(gdb) n
11         for(i=0;i<len;i++)
(gdb)

```

图 16输出变量len的值，然后单步执行


```

(gdb) n
11         for(i=0;i<len;i++)
(gdb) print rev_string[2]
$5 = 108 'l'
(gdb) n
12             rev_string[len-i]=str[i];
(gdb) n
11         for(i=0;i<len;i++)
(gdb) print rev_string[1]
$6 = 111 'o'
(gdb) n
13             rev_string[len+1]='\0';
(gdb) n
14             printf("the reverse string is%s\n",rev_string);
(gdb) print rev_string[0]
$7 = 0 '\000'
(gdb) c
Continuing.
the reverse string is
[Inferior 1 (process 18485) exited normally]
(gdb) █

```

图 17 继续单步执行，然后查看rev_string的值，直到结束

四、实验数据及结果分析

1. 步骤1中gcc的编译结果和程序的运行结果展示与说明

```

JY5@JY5:~/Desktop/test/test1$ gcc -c chang.c -o chang.o
JY5@JY5:~/Desktop/test/test1$ ar rcs libjy.so chang.o
JY5@JY5:~/Desktop/test/test1$ █

```

图 18 编译chang.c生成目标文件chang.o，将这个目标文件添加到libjy库中



图 19 首先创建了/home/test/lib16目录，然后将libjy这个库文件复制到了这个目录下

```

JY5@JY5:~/Desktop/test/test1$ gcc main.c -L /home/test/lib16 -ljy -o main
JY5@JY5:~/Desktop/test/test1$

```

图 20 编译main.c文件链接一个名为libjy的共享库，生成一个可执行文件

```

JY5@JY5:~/Desktop/test/test1$ ./main
请输入一个字符串: da454HUG
原始字符串: da454HUG
转换之后的字符串: DA454hug
JY5@JY5:~/Desktop/test/test1$ █

```

图 21 使用libjy共享库实现的字符串大小写转换功能

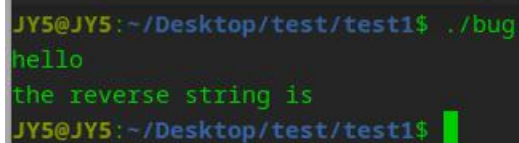
2. gdb程序调试前存在的问题和GDB调试后的原因分析及与问题修正后的结果对比

gdb程序调试前存在的问题:

数组越界, 在填充 rev_string 时, 索引计算错误导致数组越界。

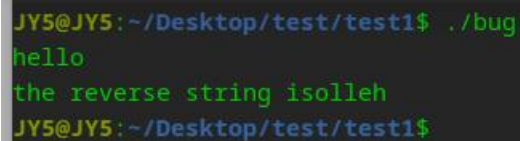
GDB调试后的原因分析及与问题修正后的结果对比:

字符串逆序问题: 通过GDB单步执行时, 可以看到程序在逆序字符赋值时, 字符存储位置是错误的。使用 print 观察 rev_string 内容后发现并不是预期的逆序字符串。



```
JY5@JY5:~/Desktop/test/test1$ ./bug
hello
the reverse string is
JY5@JY5:~/Desktop/test/test1$
```

图 22 问题修正前运行结果



```
JY5@JY5:~/Desktop/test/test1$ ./bug
hello
the reverse string isolleh
JY5@JY5:~/Desktop/test/test1$
```

图 23 问题修正后运行结果

通过GDB调试, 发现了程序中字符数组索引和结束符设置的错误, 并进行修正。

修正后的代码可以正确地输出逆序字符串

五、总结

实验成绩评定表

序号	考核项目	分值分布	成绩
1	出勤与表现	10	
2	实验任务完成情况	40	
3	实验报告质量	50	
总分			
指导教师签字			刘建明 张翰茹

备注：考核项目及分值分布应依据教学大纲确定。